MALWARE ANALYSIS LAB | REVIEW 1: PROPOSAL

Static and Dynamic Analysis of Malicious npm Packages for Behavioral Detection

Siri S Gowda | 23BCI0010

B.Tech CSE (Information Security), VIT Vellore

Domain: Software Supply Chain Security / Malware Analysis

Analysis type: static and dynamic



NPMShield

proposed detection pipeline

Problem statement

A normal npm project pulls in hundreds of packages and nobody reads all of them. Install scripts such as preinstall and postinstall run code on the developer's machine the moment a package is installed, before it is ever imported.

A package can pass both kinds of check and still be malicious.



Static-only tools

Obfuscated code and payloads decoded at runtime slip past pattern matching.



Dynamic-only tools

Many packages look for CI or sandbox signs and stay quiet, or wait before acting.

Motivation

2018

event-stream

A new maintainer added the flatmap-stream dependency, which carried code aimed at a bitcoin wallet app.

2021

ua-parser-js

The maintainer's account was hijacked. New versions installed a cryptominer and a password stealer.

2025

Shai-Hulud

A self-replicating worm stole tokens from developer machines and used them to republish infected packages.

The registry holds millions of packages, so manual review cannot keep up. npm audit only reports known vulnerabilities, not malicious intent.

Objectives

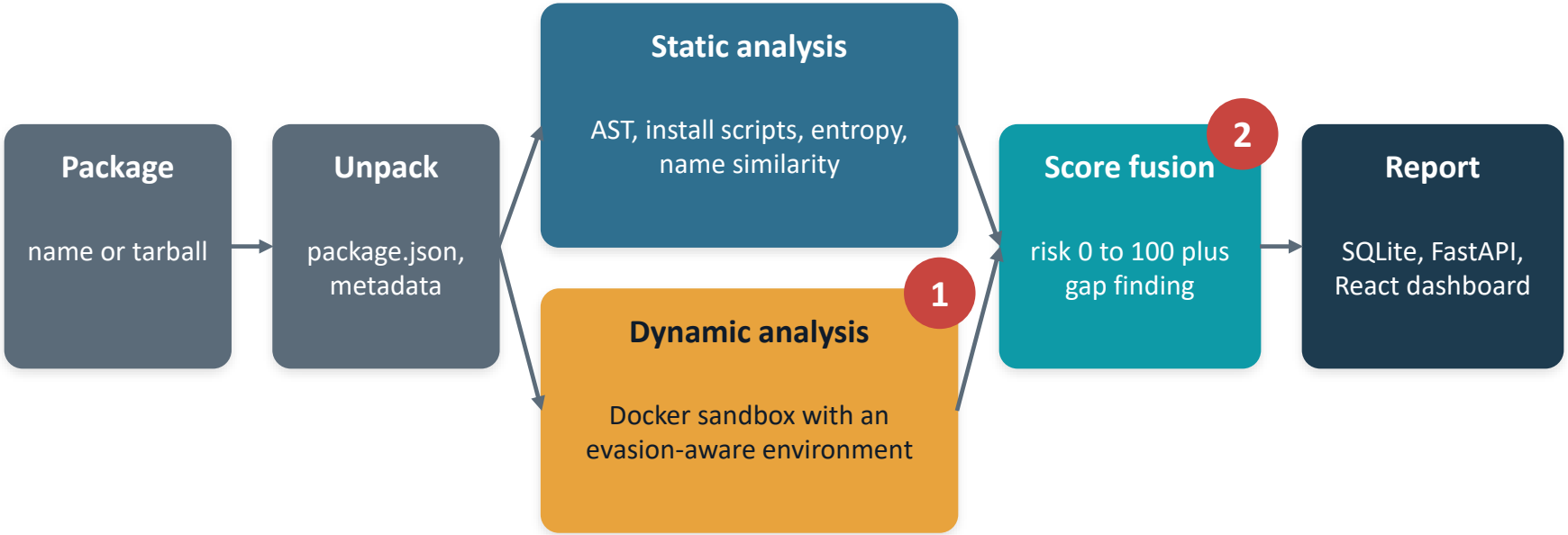
1. Collect real malicious npm packages and a benign set for comparison.
2. Build a static analyzer for install scripts, code structure, obfuscation and metadata.
3. Build a sandboxed dynamic analyzer that records file, process and network activity.
4. Make the sandbox resist evasion so dormant payloads actually run.
5. Fuse both results into one risk score and report where they disagree.
6. Extract IoCs and compare detection against static-only and dynamic-only baselines.

Existing system

Tool	What it does	Gap
npm audit	Checks packages against a list of known vulnerabilities	Does not look for malicious intent
GuardDog	Rule-based scan of source code and metadata	Mostly static, so obfuscated payloads can slip through
Socket	Commercial scanner that flags suspicious package behavior	Detection engine is closed and cannot be extended
OpenSSF package-analysis	Runs packages in a sandbox and records activity	Runs the package as is, with no handling of environment checks

Shared limit: one method per tool, no step that defeats evasion, and no comparison between the two analysis types.

Proposed system: NPMShield



1 Evasion-aware sandbox

2 Fused score with gap finding

Novel feature 1: evasion-aware sandbox

The package checks for	The sandbox responds
CI variables such as CI or GITHUB_ACTIONS	Runs twice: a laptop profile and a CI profile
Timers and date-based triggers	Moves the clock forward and shortens timers
Hostname, user, OS, CPU count	Presents realistic workstation values
Container traces such as /.dockerenv	Masks what it can and logs what it cannot
Credential files	Plants fake ones that hold canary strings



Canary check

Each fake credential holds a unique string. If that string shows up in outbound traffic, credential theft is confirmed with no guessing.

The report also lists every environment check the package attempted.

Novel feature 2: fused score and gap finding

	Quiet at runtime	Malicious at runtime
Clean statically	**Likely benign** Nothing suspicious either way.	**Gap: hidden payload** Points to obfuscation or packing.
Suspicious statically	**Gap: dormant code** Evasion not beaten. Rerun with another profile.	**Confirmed malicious** Both methods agree.

Fused risk score

0.4 x static score + 0.6 x dynamic score, with starting weights to be tuned on the sample set.

0 to 30 low
31 to 60 suspicious
61 to 100 malicious

A canary hit forces a malicious verdict.

Malware samples to be analyzed



Install-script droppers

Download and run a second stage



Credential stealers

Tokens, SSH keys, .env files



Typosquats

Names close to popular packages



Reverse shells

Backdoors and remote access



Cryptominers

Hidden mining at install or import



Delayed loaders

Obfuscated, decode or wait first



Worm-style packages

Republish using stolen tokens



Benign baseline

About 20 popular packages for false positives

Real samples only. No self-written simulations.

Tools required and dataset

Tools required

- VirtualBox with a Kali Linux VM
- Docker for the package sandbox
- Node.js, npm, acorn or Babel
- Python 3 analysis engine
- strace and tcpdump
- INetSim or mitmproxy
- FastAPI, SQLite, React
- Git and GitHub

Dataset and source

- DataDog malicious-software-packages-dataset
- OpenSSF malicious-packages and OSV advisories
- Vendor write-ups on named npm incidents
- Most downloaded npm packages (benign set)

Isolation: host-only network and a VM snapshot before every run.

Proposed methodology

1. Collect and label samples, record hashes
2. Unpack, read package.json and metadata
3. Static analysis on the AST
4. Dynamic run in the sandbox
5. Match canary values to captured traffic
6. Compute static, dynamic and fused scores
7. Extract IoCs
8. Store in SQLite, show on the dashboard
9. Evaluate against the two baselines

Expected outcome

- A tool that returns a verdict, a risk score and a behavior report for any npm package.
- A comparison on about 40 malicious and 20 benign packages: fused score against static-only and dynamic-only.
- A count of samples that ran their payload only because of the evasion-aware sandbox.
- An exported IoC list and a live demo.

1

Review 1

Proposal, design and sample plan

2

Review 2

Working pipeline and basic dashboard on 10 to 15 samples

3

Review 3

Full evaluation, IoCs and demo

References and sources

- DataDog, malicious-software-packages-dataset: github.com/DataDog/malicious-software-packages-dataset
- OpenSSF, malicious-packages: github.com/ossf/malicious-packages
- OSV, open source vulnerability database: osv.dev
- DataDog, GuardDog: github.com/DataDog/guarddog
- OpenSSF, package-analysis: github.com/ossf/package-analysis

Incident references (event-stream, ua-parser-js, Shai-Hulud) to be linked from vendor and maintainer write-ups.

Thank you.